Deep Learning 2

Special blocks

Machine learning

Roman Jakubíček
Department of Biomedical Engineering
Brno University of Technology

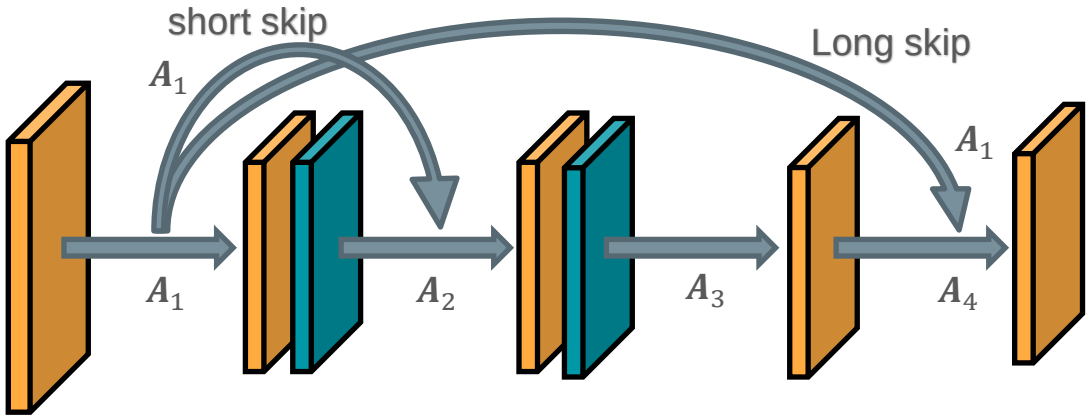
Introduction

- Other blocks and structures
  - Skip connections
  - 1×1 convolution, Bottleneck
  - Transposed and dilated convolution
  - Un-pooling layer
  - Residual block
  - Attention module

Special blocks

Skip connections

- Short or long skip connections
- Skip connections help traverse information in deep neural networks
- It is trying to find and to use the layers which are rich in features
- It provides improving the information flow between layers
- Solve vanishing or exploding problems (BatchNorm helps in depth) in ”shallow” nets



Output from l^{th} layer - A_l

… but, how do we can join data?

Skip connections

Concatenation

– in the context of programming, is the operation of joining two strings together
– joining several same sized (can be ensured) activation maps into depth as additional features
– output tensor will be expanded in depth

$$\mathrm{A}_l^* = [\mathrm{A}_1, \dots, \mathrm{A}_{l-1}] \qquad \text{e.g.} \quad \mathrm{A}_4^* = [\mathrm{A}_1, \mathrm{A}_4]$$

Additive (residual)

– add a skip-connection that bypasses the non-linear transformations with an identity function
– to adding, the tensors must be of the same-sized (it can be solved padding, interpolation or pooling)
– output will be same sized as input in depth

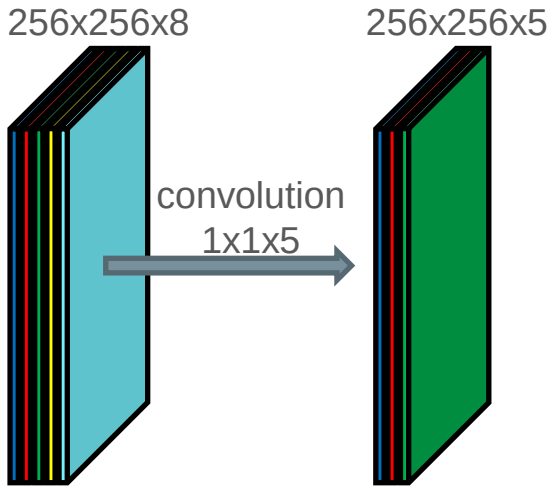
$$\mathrm{A}_l^* = [\mathrm{A}_1 + A_2 + A_{\{l-1\}}] \qquad \text{e.g.} \quad \mathrm{A}_4^* = [\mathrm{A}_1 + \mathrm{A}_4]$$

1 x 1 convolution layer

- Sometimes called “point-wise convolution”
- It is not combining spatial information, but it is combining features in depth
- The output is the same size as input in spatial domain, but the depth (number of features) can be changed.
- We obtain new activation maps (feature maps), which are a linear combination of previous features
- It has no hyper-parameters, but there are learned the weights of individual features

Why would we want 1x1 convolution?

- New combination features
- Reduce or expand features
- Speed up by decreasing the dimension
- Separable convolution (MobileNet)
- Alternative computation of FC layer (assuming 1x1xN tensor as input)

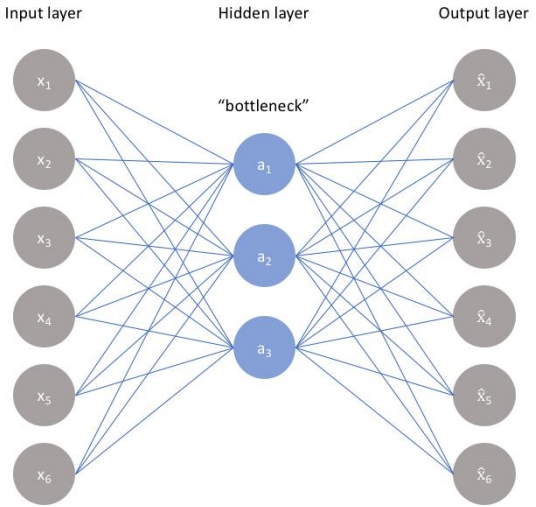


BottleNeck – autoencoders

- constrains the amount of information that can traverse the full network, forcing a learned compression of the input data (“regularization”)
- It is designed for finding the best tradeoff between accuracy and complexity (compression)
- To find (learn) weights of A(D)NN for which it applies that

$$\min(||\boldsymbol{x} - \hat{\boldsymbol{x}}||)$$

- The bottleneck is a key property/part of the network architecture, without this bottleneck, our network could easily learn to simply memorize the input values by passing these values through the network.

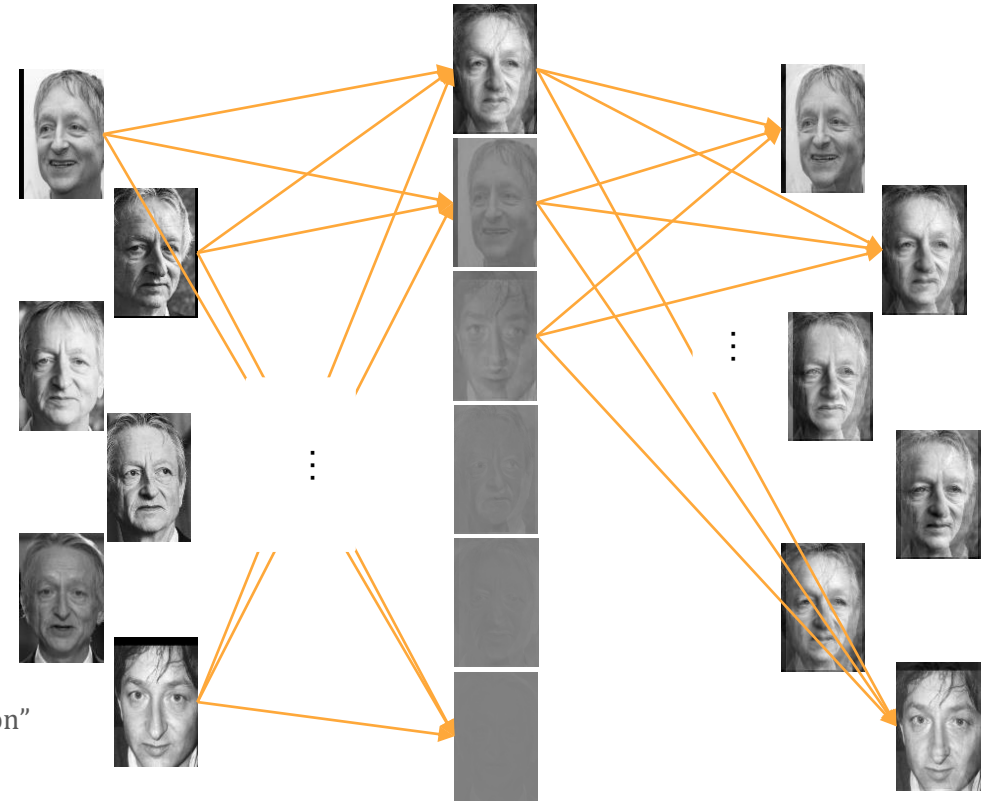


Applications:

- Dimensional reduction (compression)
- Encoder – decoder (U-net)
- Bottleneck layer (GoogLeNet)

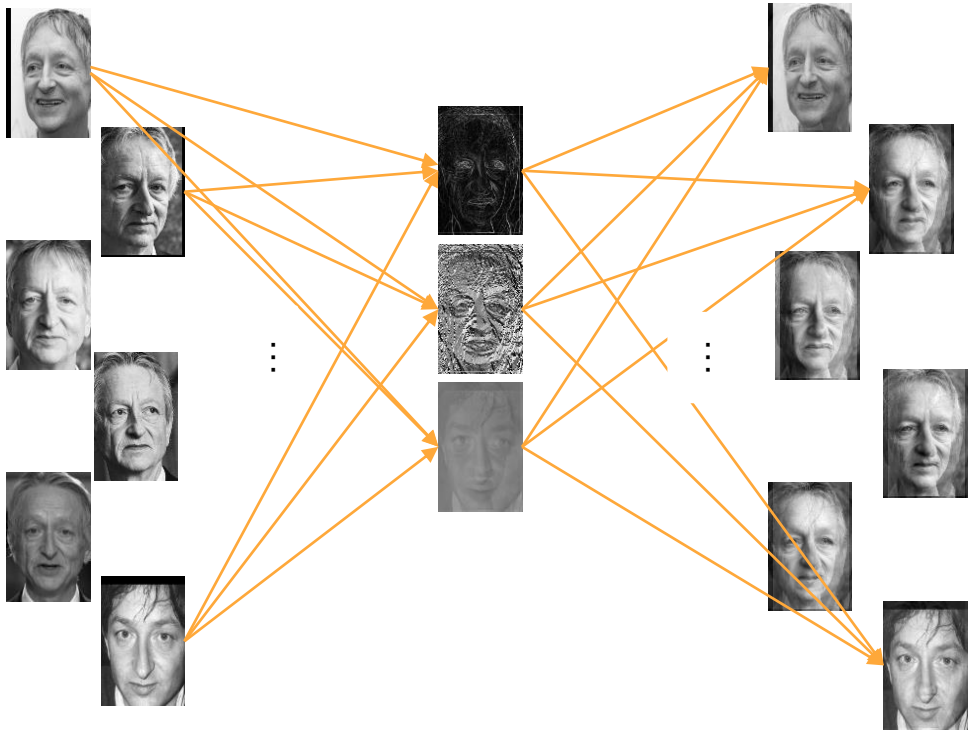
BottleNeck

Example feature reduction for face recognition by PCA

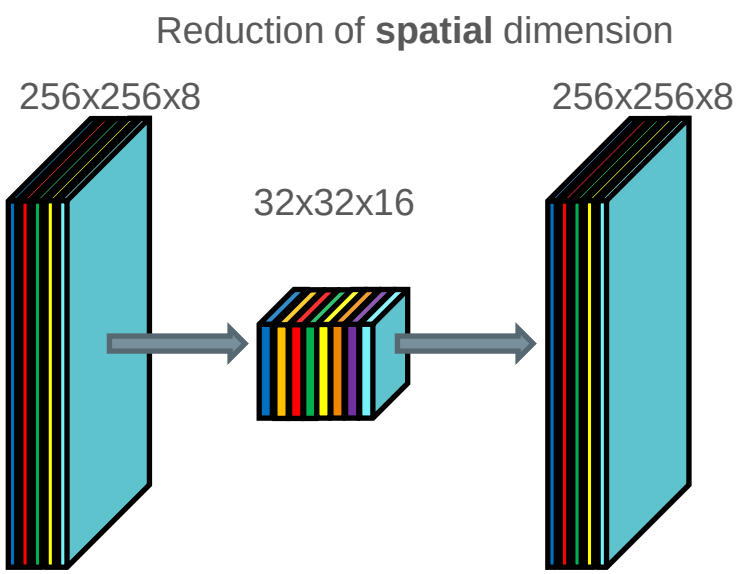


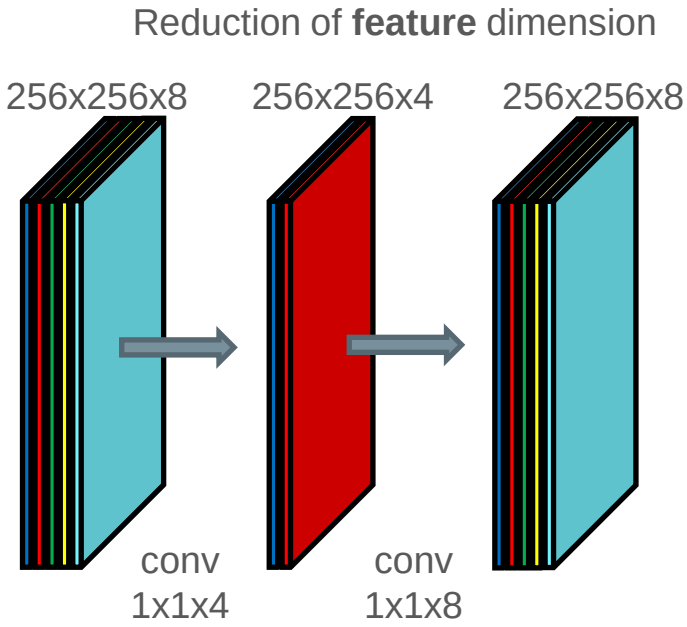
BottleNeck

Example feature (information) reduction for face recognition by NN



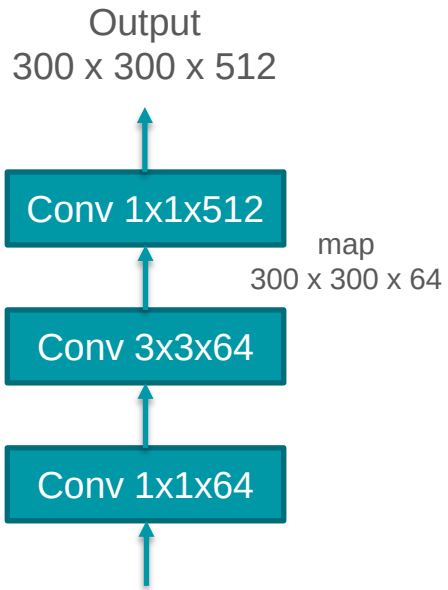
BottleNeck


Reduction of **spatial** dimension
256x256x8
32x32x16
256x256x8

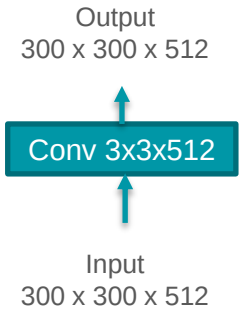

Reduction of **feature** dimension
256x256x8
256x256x4
256x256x8
conv
1x1x4
conv
1x1x8

BottleNeck layer

- for convolution neural networks
- to reduce the size of the input tensor in a convolutional layer
- lead to large saving in a computational cost
- preserves information with maximal relevance
- included in GoogLeNet (Inception V2 and V3) or DenseNet



“



”

Dilated convolution

- aims to aggregating the multi-scale contextual information

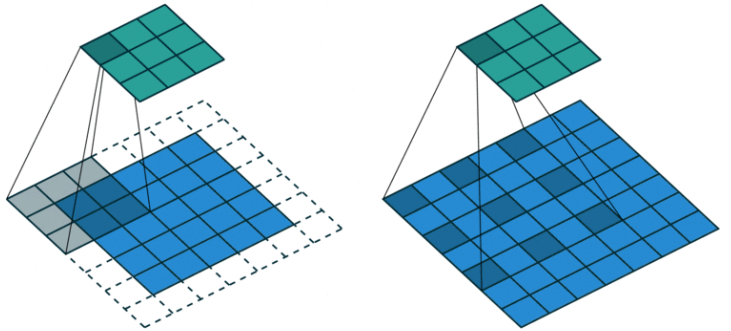
Standard convolution

$$F(\boldsymbol{p}) * k(\boldsymbol{p}) = \sum_{\boldsymbol{p}=\boldsymbol{s}+\boldsymbol{t}} F(\boldsymbol{t})k(\boldsymbol{s})$$

Dilated convolution

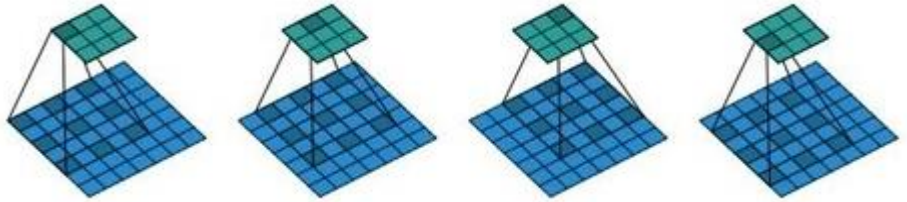
$$F(\boldsymbol{p}) *_{\delta} k(\boldsymbol{p}) = \sum_{\boldsymbol{p}=\boldsymbol{s}+\delta\boldsymbol{t}} F(\boldsymbol{t})k(\boldsymbol{s})$$

Dilation factor $\delta = 1$ $\delta = 2$



- no down-sampling
- the receptive field is larger without the need for the down and up-sampling
- analogously, the classic convolution with larger sparse kernel or pyramidal approach

Dilated convolution with δ factor = 2



Up-sampling layers

The neural networks to generate images, it usually involves up-sampling from low resolution (from depth) to high resolution (output)

Un-pooling

- suitable for resampling (up) after a max pooling layer (non-linear)

Transposed convolution

- This structure enables up-sampling matrices (features maps) from previous layers by convolution performed via matrix multiplication. Weights of convolution kernel are learned to achieved the best interpolation

Interpolation

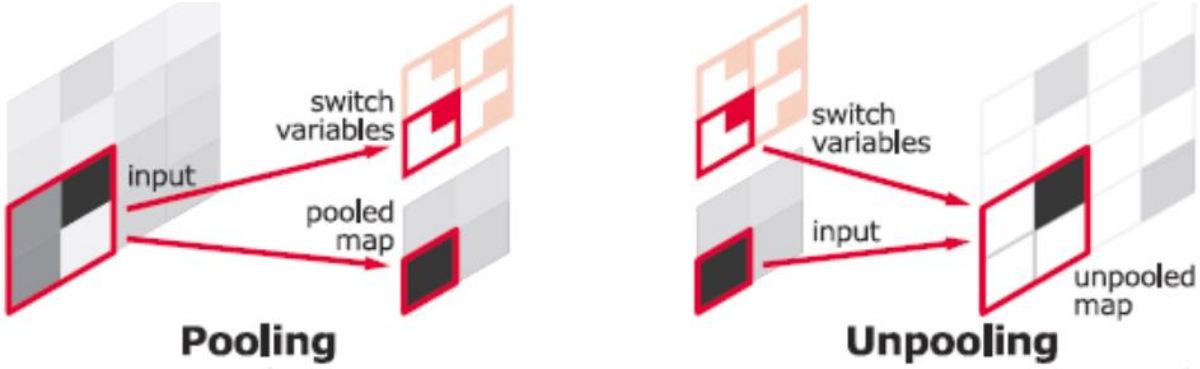
- elementary interpolation method (bilinear)
- there is nothing that the network can learn about

Un-pooling layer (max)

- the task of this layer is up-sampling input (after pooling) – reverse pooling

- to perform un-pooling, we need to remember the position of each maximum activation value when doing max pooling
- in the un-pooling layer, the max value is inserted to the remembered position
- remains values are set to zeros
- there are no parameters for learning

- in the case, average pooling is performed by an interpolation method



Transposed convolution

- Sometimes wrongly called “deconvolution”
- It is using a computation of convolution via matrix multiplication…

$$G(\boldsymbol{p}) = F(\boldsymbol{p}) * k(\boldsymbol{p}) = \sum_{p=s+t} F(\boldsymbol{t})k(\boldsymbol{s}) \quad \Rightarrow \quad vec(\boldsymbol{G}) = \boldsymbol{C}\, vec(\boldsymbol{F})$$

… standard convolution with position vector $\boldsymbol{p}$

… where $\boldsymbol{G}$ is output matrix (convoluted matrix), $\boldsymbol{C}$ is **convolution matrix** and $\boldsymbol{F}$ is matrix of data

Note:

Then, inverse convolution can be derived to obtaining the matrix $\boldsymbol{F}$ from $\boldsymbol{G}$ and $\boldsymbol{C}$ as follows

$$vec(\boldsymbol{F}) = \boldsymbol{C}^{-1} vec(\boldsymbol{G})$$

… it is not possible, because we cannot determine inverse convolution matrix $\boldsymbol{C}$.

But we can use the convolution matrix to compute convolution (even stride-conv) and then use it for the up-sampling of $\boldsymbol{G}$ to the same size as matrix $\boldsymbol{F}$ (input to convolution).

Transposed convolution

$$vec(\boldsymbol{G}) = \boldsymbol{C}\, vec(\boldsymbol{F})$$

- Up-sampling method using **convolution matrix** with the specific size
- We can eliminate the impact of stride >1 in convolution layers
- Via matrix multiplication, we can effectively compute convolution with stride >1

Weights of kernel 3x3 ($\boldsymbol{K = 3}$)

$$\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \\ w_{3,1} & w_{3,2} & w_{3,3} \end{bmatrix}$$

Input image $\boldsymbol{M \times M}$

After padding

$$\begin{bmatrix} x_{1,1} & \cdots & x_{1,M} \\ \vdots & \ddots & \vdots \\ x_{M,1} & \cdots & x_{M,M} \end{bmatrix} \Rightarrow \begin{bmatrix} x_{1,1} \\ x_{1,2} \\ \vdots \\ x_{1,M} \\ x_{2,1} \\ \vdots \\ x_{M,M} \end{bmatrix} \quad M^2$$

$\boldsymbol{F}$

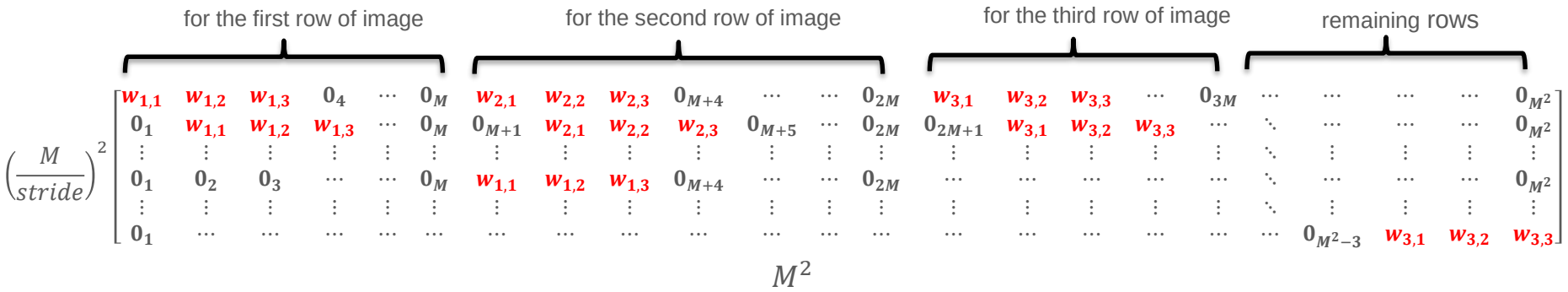
$vec(\boldsymbol{F})$

- To preserve the same size of output, we can apply a padding on input image $N \times N$
- We obtain a new size of image $M \times M$, where $M = N + (K - 1)$

Convolution matrix C

- rearranging of kernel to same size as input (after padding) image, thus $\boldsymbol{M \times M}$

For standard convolution with stride = 1

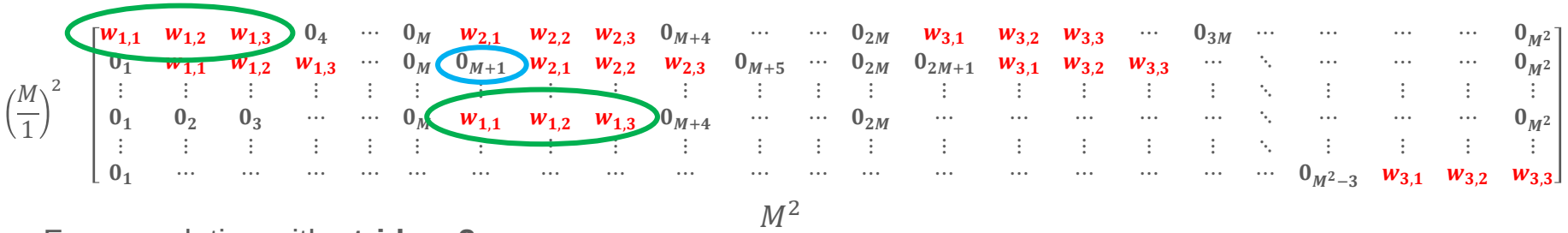


Each row of the matrix C represents a convolution calculation for single specific translation of kernel

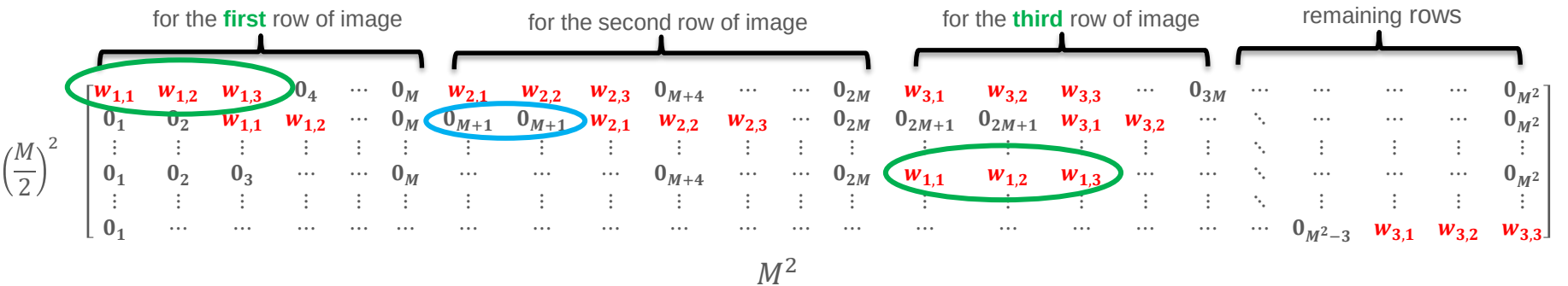
Note: $M = N + (K - 1)$, N is a size of input image, K is size of kernel and M is size image after padding

Convolution matrix C

For **standard** convolution with **stride = 1**



For convolution with **stride = 2**

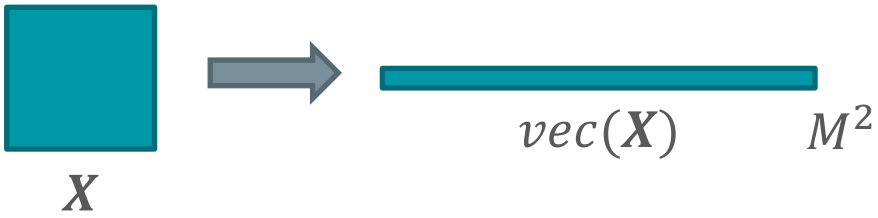


If stride = 2, the column of C for the second row (and every other row) of the image is equal to 0

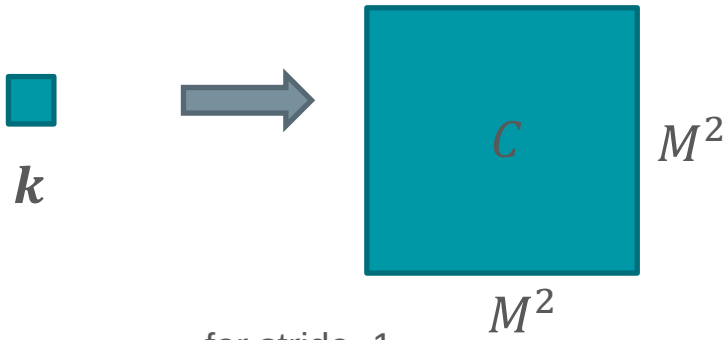
Note: $M = N + (K - 1)$, N is a size of input image, K is size of kernel and M is size image **after padding**

Transposed convolution

1. Vectorization of activation map (image) sized $M \times \mathrm{M}$

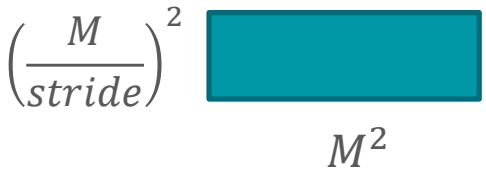


2. Creation of sparse convolution matrix **C** from kernel



for stride=1

Convolution matrix for conv with stride=2

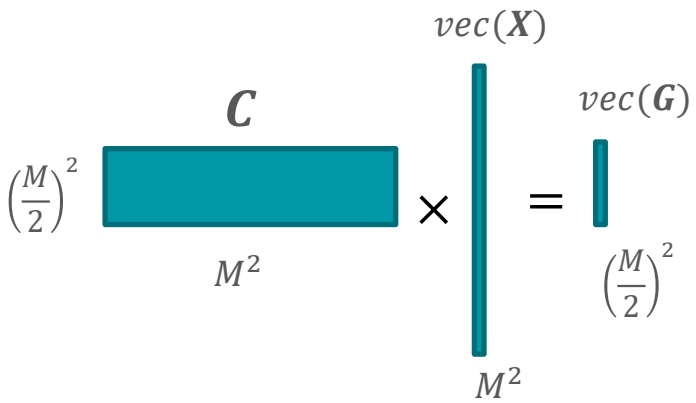


Transposed convolution

Then, we easily can up-sample activation maps into the original size (before convolution layer with stride >1, which caused its sub-sampling).
We need only the size of the stride and size of sub-sampled map. We create an initial convolution matrix **K** with the same size as matrix **C**, whose values will be learned.

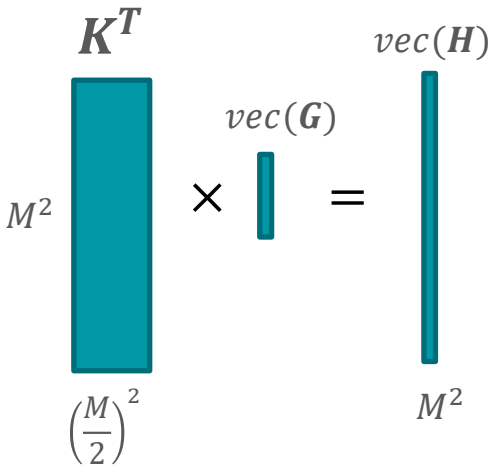
Convolution with stride=2

$$vec(\boldsymbol{G}) = \boldsymbol{C}\, vec(\boldsymbol{X})$$



Transposed convolution with stride=2

$$vec(\boldsymbol{H}) = \boldsymbol{K}^T\, vec(\boldsymbol{B})$$



Residual block

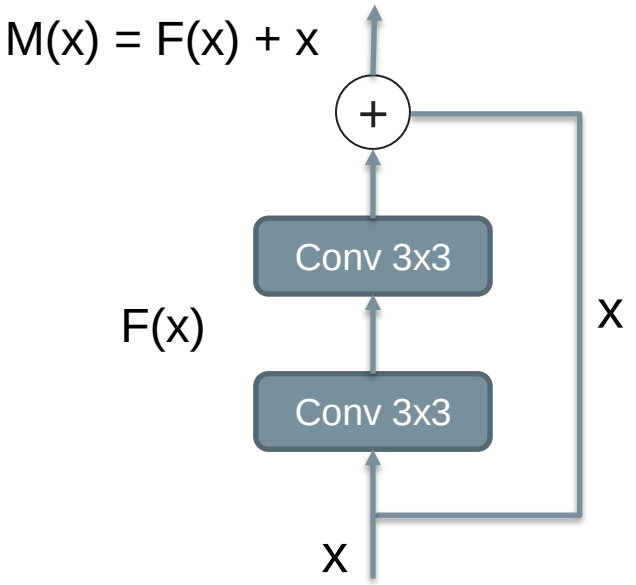
- The deeper model should be able to perform at least as well as the shallower model
- but deeper models are harder to optimize
- It has two branches flowing into additive connection (identity and convoluted identity)

- Thus, the weights of conv layers are adapted this way, that the output from them will “residuum” defined as follows

$$F(x) = M(x) - x$$

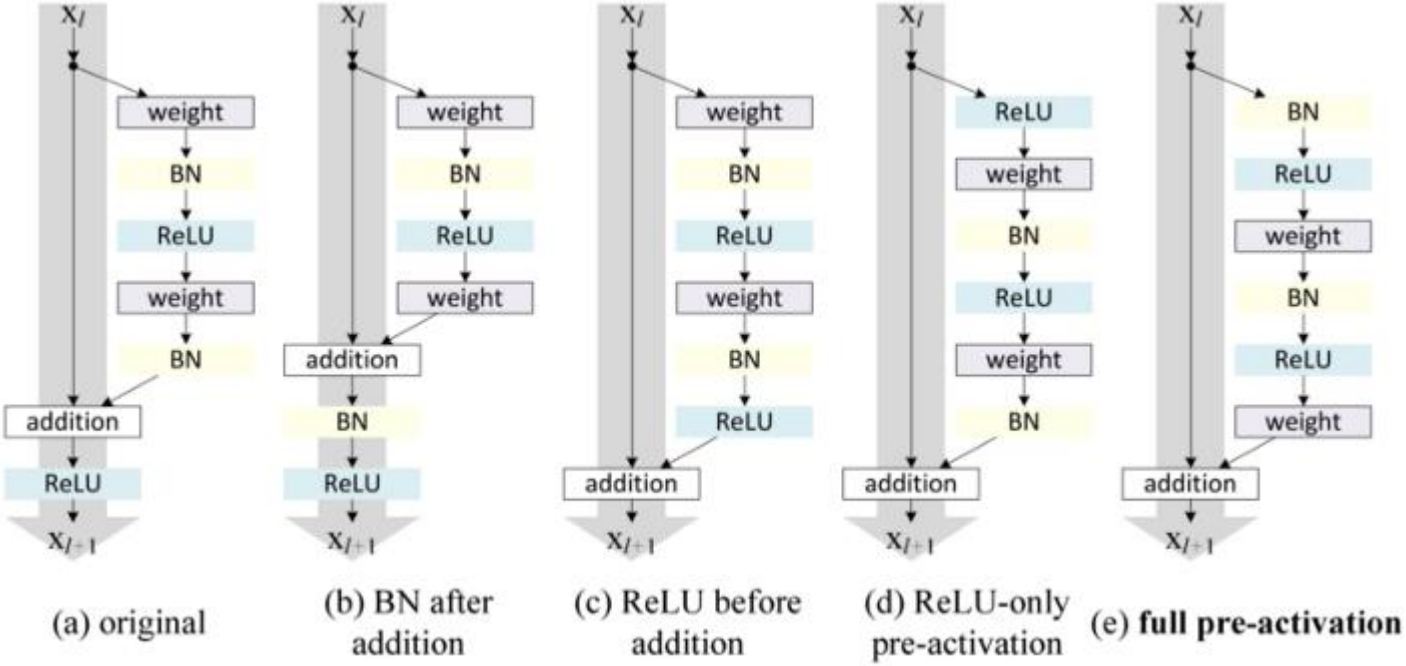
- In other words, this block can take the output from a series of conv layers or just original input (skip layer)

- Avoids vanishing gradients
- May be able to learn simple function (identity function)



Residual block

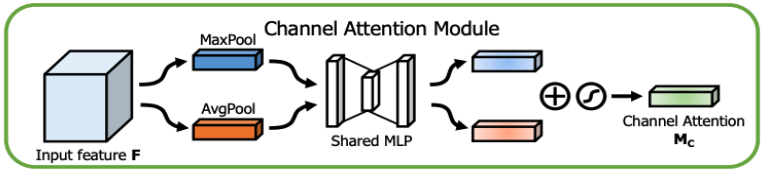
- There are many variations of block structure
- Can also be included 1x1 convolution in the identity branch



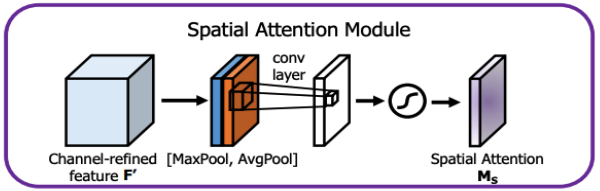
Attention module

- The main idea is, in the first stage, the network pays attention to the global (overall) context
- Then, it can focus on details in the full resolution
- For tasks which require a global perceiving, not point-based approach

- Fully connected–based networks
- Residuals blocks
- The decoder uses an activation map from each level



- **Attention map** – a weight matrix that defines the area of interest (localization/feature) in the activation map.



- Provides contextual information about the problem
- Useful in recurrent nets (eliminate vanishing problem)
- For segmentation task – spatial information
- Basic unit of special network called “Transformers”

- high computational complexity (fully connected)